

Non-Prehensile Throwing: A Reinforcement Learning Perspective

Anonymous Author(s)*¹

Abstract—Robotic throwing enables fast object transport and extends a robot’s reachable workspace beyond traditional pick-and-place. While prehensile (grasp-based) throwing works well for graspable items, non-prehensile (grasp-free) throwing is better suited for large, heavy, and/or deformable objects. Existing approaches rely on model-based optimization with simplified contact models (e.g., dynamic grasping) and low-dimensional trajectory parameterizations, which limit solution quality and reachable workspace. We propose a reinforcement learning approach that additionally leverages sliding and rolling contact modes and directly optimizes joint-space trajectories without analytical contact models or custom parameterizations. The Markov Decision Process (MDP) is formulated as a dynamical system that evolves the robot’s joint state conditioned on the throwing target, object model, and initial configuration. Joint-jerk trajectories are planned offline at a low control rate and upsampled into smooth, high-rate velocity commands for deployment. For sim-to-real transfer, we minimize the robot-dynamics gap through minimum-jerk system identification and train uncertainty-aware policies to mitigate object-modeling errors, particularly sensitivity to dynamic friction. In simulation, the policy achieves 99% success across thousands of configurations and generalizes to unseen objects. Sensitivity analysis shows robustness to mass uncertainty but high sensitivity to dynamic friction, consistent with the sliding-based release mechanism. Deployed zero-shot on a UR5e operating near its physical limits (5 m/s end-effector velocity), our method throws diverse objects—including heavy (790 g) and large ($20 \times 20 \times 28$ cm) items—to targets up to 350 cm distance or 180 cm elevation, achieving a 97% real-world success rate.

I. INTRODUCTION

Throwing enables robots to transport objects rapidly and extend their effective workspace beyond quasi-static pick-and-place. As a result, robotic throwing has received increasing attention, with two main modes considered in the literature: prehensile (using a gripper) and non-prehensile (using a tray).

Prehensile throwing performs well for small objects [1, 2], where a trajectory is optimized to reach a desired end-effector pose and velocity before gripper release. Larger or heavier objects can be manipulated using oversized industrial grippers [3] or bi-manual systems [4], but these systems are difficult to coordinate at high velocities and less precise for smaller items. For deformable objects, prehensile throwing can induce grasp instability and uncertain release timing [2].

Non-prehensile throwing offers an alternative that naturally accommodates large, heavy, or deformable objects. However, without grasp forces, the system must satisfy kinodynamic constraints to avoid unintended sticking, sliding, or rolling. Prior work enforces these constraints through dynamic grasping [5, 6, 7, 8], slip-less rolling for spherical objects [9], or mechanically constrained setups [10]. While these approaches provide valuable modeling insight, their



Fig. 1: *NP-Throw* enables planar non-prehensile throwing of generic objects to distant and elevated targets. Its sliding-based release mechanism avoids the infeasible fast decelerations used by dynamic-grasping approaches. **Top** (200 cm, 15 cm): Multiple objects, including a tall chips can and a small cube. **Middle** (250 cm, 105 cm): Large box. **Bottom** (300 cm, 185 cm): Heavy wood block. Videos: <https://tinyurl.com/np-throwing>.

deployment relies on simplified contact models and low-dimensional trajectory parameterizations. Moreover, optimization is typically performed via sequential quadratic programming (SQP), which is sensitive to initialization and prone to local minima and solver failures—ultimately restricting performance and limiting applicability to a small set of carefully designed scenarios.

RL perspective. Reinforcement learning offers a complementary approach that does not require fixed contact-mode assumptions or hand-designed trajectory parameterizations. In a physics simulator, RL can discover additional hybrid sliding/rolling interactions and generate explicit joint-space trajectories by training over thousands of randomized environments—overcoming the expressiveness limits of analytical models and the brittleness of SQP solvers.

This work. We propose *NP-Throw*, an RL-based non-prehensile throwing policy. The MDP is formulated as a dynamical system defined over implicit robot-state observations; the policy outputs joint-jerk commands at a low rate,

*Corresponding author, reachable at Anonymous@Author.com

¹Anonymous Author is with Anonymous University

which are upsampled into smooth velocity trajectories for deployment. To bridge the sim-to-real gap, we (i) calibrate robot dynamics via minimum-jerk system identification, and (ii) train uncertainty-aware policies to address object-modeling errors—especially sensitivity to dynamic friction.

Key results. In simulation, the policy attains $\sim 99\%$ success across thousands of in-distribution configurations and generalizes effectively to unseen YCB objects [25]. Sensitivity analysis shows robustness to mass-estimation errors but high sensitivity to dynamic-friction mismatch, consistent with the policy’s reliance on sliding-based release. When deployed on real hardware, *NP-Throw* achieves an average success rate of **97%** across five challenging objects and long-range targets (up to 350 cm distance or 185 cm elevation), operating near the robot’s physical limits (5 m/s end-effector velocity). Sample planar throwing trajectories are shown in Fig. 1

Contributions. We present an RL formulation for planar non-prehensile throwing that learns hybrid contact behaviors without fixed contact-mode assumptions or trajectory parameterizations. We provide extensive simulation studies analyzing policy’s performance, out-of-distribution generalization, and sensitivity to modeling errors. We introduce a practical sim-to-real pipeline—leveraging uncertainty-aware training and minimal dynamics tuning—that enables reliable zero-shot deployment on real hardware.

Code: <https://tinyurl.com/np-throwing>

II. RELATED WORKS

Non-prehensile Throwing. To simplify trajectory optimization, prior work often adopts low-dimensional trajectory parameterizations—such as B-splines [5, 8], radial basis functions [9], or polynomials [10]—which restrict the feasible trajectory space and can yield suboptimal solutions. To handle hybrid dynamics, most approaches assume a single contact mode (e.g., dynamic grasping [5, 7, 8]) or separately optimize motion primitives with predefined mode transitions [6, 9], further limiting exploration. Experimental demonstrations are typically constrained to a single object type (e.g., cuboid or sphere) and specialized setups such as tilted air-hockey tables [6, 7, 9].

Prehensile Throwing. To address the sensitivity of model-based optimization to initialization, several works use offline-generated datasets to improve and accelerate trajectory optimization. In [11], a dataset mapping release states to robot states is combined with a throwing manifold to obtain multiple *good* initial guesses. Nearest-neighbor retrieval in landing or release space provides approximate trajectories, and failed executions are reused to reduce the sim-to-real gap [12, 13]. Differentiable motion-manifold primitives are learned and latent samplers are refined with real data to generate improved initial trajectories [14, 15]. Extending these dataset-driven strategies to non-prehensile throwing inherits limitations of the underlying data-generation process—namely, low-dimensional parameterization and single-mode contact assumptions. The additional kino-dynamic constraints in non-prehensile settings also make trajectory generation for generic objects substantially more computationally expensive. Although

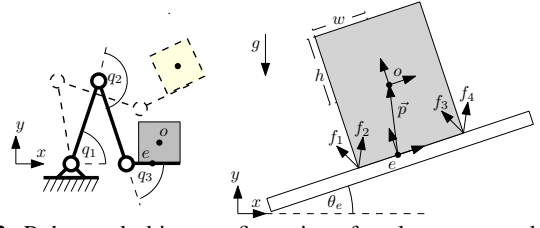


Fig. 2: Robot and object configurations for planar non-prehensile throwing.

random throwing trajectories bypass optimization and reduce dataset cost [12, 16], they would fail to adequately cover the reachable workspace in non-prehensile scenarios.

Learning-based Prehensile Throwing. End-to-end learning offers an alternative. Self-supervised residual policy [1] corrects release pose and velocity directly on real hardware, while [17] learns dart throwing via off-policy RL with constrained action parameterization. Simulation-trained throwing policies can transfer zero-shot to real systems [18]. However, reducing the action space to a release-state parameterization is insufficient for non-prehensile throwing, where the release state emerges from long-horizon contact-rich dynamics. Nonetheless, large-scale RL with explicit joint-space actions, as shown in dexterous manipulation [19], has provided a viable framework for learning complex manipulation tasks.

III. PROBLEM FORMULATION

We address the problem of motion planning for non-prehensile throwing. Owing to equivariance in the throwing direction, we focus on planar throwing. Given the object model, the initial robot/object configurations, and a throwing target, a throwing trajectory is generated offline and executed without feedback on the object state. Fig. 2 illustrates the robot and object configurations. Without loss of generality, we consider a fixed-base robot with three coplanar revolute joints ($\mathbf{q} = [q_1, q_2, q_3]$). A uniform-density cuboid object is described by its mass m , height $2h$, width $2w$, and static/dynamic friction coefficients μ_s, μ_k (collectively $\beta = [m, h, w, \mu_k, \mu_s]$). The planar target is specified by the horizontal distance r_t and elevation z_t in the world frame.

Next, we introduce a simplified dynamic-grasping model to (i) motivate the MDP design choices, (ii) interpret later key empirical findings, and (iii) highlight why analytical approaches struggle with hybrid contact dynamics.

A. Dynamic-Grasping Throwing Model

Following [20], we adopt the Newton–Euler balance ($\mathbf{w}_{GI} + \mathbf{w}_C = 0$) to derive a simplified planar model, where $\mathbf{w}_{GI}$ and $\mathbf{w}_C$ denote the gravito-inertial and contact wrenches in the object frame. The object lies on a flat tray, with its pose relative to the end-effector given by $\mathbf{p} = [p_x, p_y]$. The end-effector state includes planar pose $\mathbf{x}_e = [x_e, y_e, \theta_e]$, velocities $\mathbf{v}_e$ and $\omega_e = \dot{\theta}_e$, and accelerations $\dot{\mathbf{v}}_e$ and $\dot{\omega}_e$.

Under sticking or sliding, the object’s rotational motion matches the end-effector ($\theta_o = \theta_e$, $\omega_o = \omega_e$, $\dot{\omega}_o = \dot{\omega}_e$). Its linear velocity and acceleration follow

$$\mathbf{v}_o = \mathbf{v}_e + \omega_e \times \mathbf{p}, \quad \dot{\mathbf{v}}_o = \dot{\mathbf{v}}_e + \dot{\omega}_e \times \mathbf{p} + \omega_e \times \dot{\mathbf{p}},$$

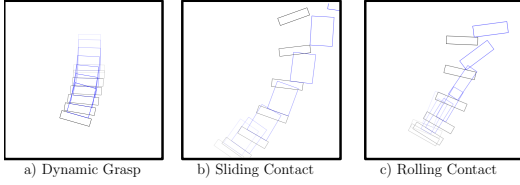


Fig. 3: Contact modes exhibited during *NP-Throw*. The end-effector and object are shown in black and blue, respectively.

yielding

$$\mathbf{w}_{\text{GI}} = \begin{bmatrix} m(\ddot{x}_e + g \sin \theta_e - \ddot{\theta}_e p_y) \\ m(\ddot{y}_e - g \cos \theta_e + \ddot{\theta}_e p_x + \dot{\theta}_e \dot{p}_x) \\ I \ddot{\theta}_e \end{bmatrix}. \quad (1)$$

Contact force is approximated using two point contacts. Each force is expressed as a positive combination of friction-cone edge vectors: $\mathbf{f}_{i_l} = [-\mu, 1] \mathbf{f}_{i_l}$ and $\mathbf{f}_{i_r} = [\mu, 1] \mathbf{f}_{i_r}$ with $f_{i_l}, f_{i_r} \geq 0$. The resulting wrench is

$$\mathbf{w}_C = [F_t, F_n, F_t h + \tau]^T, \quad (2)$$

where normal contact force is $F_n = f_1 + f_2 + f_3 + f_4$, tangential contact force is $F_t = -\mu(f_1 + f_3) + \mu(f_2 + f_4)$, and contact moment is $\tau = (-f_1 - f_2 + f_3 + f_4)w$.

Because $f_i \geq 0$, the following friction-cone constraints enforce anti-slip and anti-roll behavior:

$$\mu F_n \geq |F_t|, \quad w F_n \geq |\tau|. \quad (3)$$

Challenges of Analytical Models. Throughout the trajectory (Fig. 3), the object can exhibit dynamic grasp ($f_{1,2,3,4} > 0$), sliding ($f_{1,3} = 0$ or $f_{2,4} = 0$), rolling ($f_{1,2} = 0$ or $f_{3,4} = 0$), or mixed modes. The earlier simplified model assumes no rolling and fixed contact locations on the object—assumptions that break as soon as significant sliding occurs. Accurately modeling these hybrid transitions analytically and optimizing over them is therefore non-trivial. While trajectory-optimization methods [5, 6, 7, 8] typically assume a single contact mode or prescribe a fixed sequence, our objective is to learn a policy that implicitly controls hybrid contact dynamics and the necessary mode transitions for optimal task performance.

Insights for MDP Design. This simplified formulation provides three key insights for policy design:

- The object state is determined by the end-effector trajectory and object model in absence of disturbances,
- A minimal object model consists of mass (inertia), geometry (width/height), and contact friction (static/dynamic).
- Controlling the end-effector at the acceleration or jerk level is essential for smooth contact-mode transitions.

IV. REINFORCEMENT LEARNING PERSPECTIVE

We first introduce the proposed Markov Decision Process (MDP) for offline motion planning, followed by the procedure for successful sim-to-real transfer.

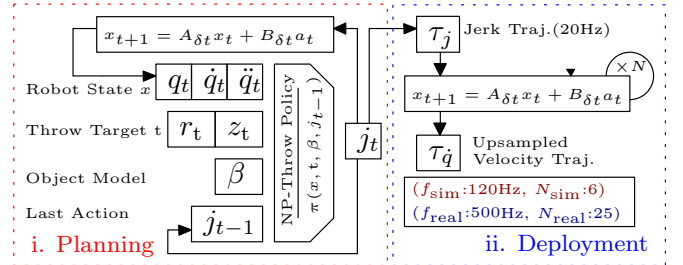


Fig. 4: Throwing trajectory planning using the *NP-Throw* policy. Given an object model β , throwing target $\mathbf{t} = (r_t, z_t)$, and initial joint configuration $\mathbf{x}$, the jerk trajectory is generated offline by recursively evaluating the policy. The resulting trajectory is upsampled into high-rate velocity commands for execution in simulation and on real hardware.

A. Markov Decision Process Formulation

The objective of reinforcement learning (RL) is to learn a policy π^* that maps a state $s_t \in \mathcal{S}$ to an action $a_t = \pi^*(s_t) \in \mathcal{A}$, maximizing the expected discounted return over a finite horizon, $R_t = \mathbb{E} \left[\sum_{i=t}^T \gamma^{i-t} r_{i+1} \right]$, where T is the horizon length and $\gamma \in [0, 1)$ is the discount factor. We consider a fully observed, deterministic MDP in which the initial state $s_0 \sim \mathcal{S}_0$ evolves according to a desired dynamical system:

$$\mathbf{s}_{t+1} = f(\mathbf{s}_t, a_t) = f(\mathbf{s}_t, \pi^*(s_t)).$$

The transition function f is defined solely over the robot state. Although the object state is not explicitly included, in the absence of disturbances it is implicitly determined by the robot's trajectory and the object model. The overall architecture is illustrated in Fig. 4.

1) **Initial State Distribution $\mathcal{S}_0$:** The initial state includes the robot state $\mathbf{x}$, object model parameters β , and the target (r_t, z_t) . Table I summarizes the sampling distributions used for throwing-configuration randomization. These ranges were selected empirically based on our robot, tray design, and object set, and should be adapted appropriately for other systems. The object is modeled as a cuboid, and we randomize its mass, width, height, and static/dynamic friction coefficients. The initial robot configuration is randomized such that the tray remains horizontal and points along the positive x -direction (Fig. 2). All joint velocities, accelerations, and jerks are initialized to zero. The throwing target is sampled to span the reachable distance and elevation enabled by hardware limits (5 m/s maximum end-effector velocity for UR5e robot). To ensure full observability, the object is always initialized at the center of the tray, and its center of mass (COM) is assumed to coincide with the geometric center. Additional randomization may be incorporated towards further robustness.

2) **Observation Space:** The observation space comprises the robot state (position, velocity, acceleration) $\mathbf{x}_t = [\mathbf{q}_t, \dot{\mathbf{q}}_t, \ddot{\mathbf{q}}_t]$, the object model $\beta = \{m, w, h, \mu_s, \mu_k\}$, the throwing target (r_t, z_t) , and the last action a_{t-1} . We also ablated the use of extended histories $\mathbf{x}_{t-k:t}$ and $\mathbf{a}_{t-k-1:t-1}$, which did not improve performance. The robot state $\mathbf{x}_t$ is updated recursively given the jerk action $\ddot{\mathbf{q}}_t$, without accessing ground-truth simulated or real state. In absence of disturbances, the object state can be implicitly inferred by the

TABLE I: Sampling ranges for randomizing the initial state s_0 .

Property	Unit	Distribution
Object Model β		
Width	cm	$\mathcal{U}(5, 20)$
Height	cm	$\mathcal{U}(5, 25)$
Mass	kg	$\mathcal{U}(0.1, 1.0)$
Static/Dynamic Friction	–	$\mathcal{U}(0.2, 1.0)$
Throwing Target		
Distance	cm	$\mathcal{U}(150, 350)$
Elevation	cm	$\mathcal{U}(0, 200)$
Joint Configuration q		
Joint 1 (q_1)	rad	$-\pi/4 \pm \mathcal{U}(-\pi/4, \pi/4)$
Joint 3 (q_3)	rad	$-\pi/4 \pm \mathcal{U}(-\pi/4, \pi/4)$
Joint 2 (q_2) ^a	rad	$q_2 = -(q_1 + q_3)$

^a Ensures a horizontal tray at initialization for UR5e robot.

policy knowing the end-effector trajectory and object model. The object model β follows the planar mechanics derived in the previous section. To improve learning stability and performance, we adopt an asymmetric actor-critic architecture: the critic receives privileged information, including end-effector and object states (pose and velocity), as well as a binary indicator of whether the object has been released (i.e., zero contact force with the tray). This indicator, $i_{\text{thrown},t}$, is used in the reward and is important for accurate value estimation.

3) *Action Space*: While a high control frequency can yield smooth trajectories and improved tracking, it also increases trajectory length and slows simulation and learning. We therefore design the action space to ensure smoothness with minimal horizon length for task success. Specifically, we use jerk-based actions ($a_t = \ddot{q}$) together with up-sampling for robust trajectory tracking. Given the joint jerk, the robot joint state $x_t = [q_t, \dot{q}_t, \ddot{q}_t]$ is updated as

$$x_{t+1} = Ax_t + Ba_t, \quad (4)$$

where

$$A = \begin{bmatrix} \mathbf{I}_3 & \delta t \mathbf{I}_3 & (1/2)\delta t^2 \mathbf{I}_3 \\ \mathbf{0}_3 & \mathbf{I}_3 & \delta t \mathbf{I}_3 \\ \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{I}_3 \end{bmatrix}, \quad B = \begin{bmatrix} (1/6)\delta t^3 \mathbf{I}_3 \\ (1/2)\delta t^2 \mathbf{I}_3 \\ \delta t \mathbf{I}_3 \end{bmatrix}.$$

Here, $\mathbf{I}_3$ is the 3×3 identity matrix, and δt is the integration step (with control frequency $f = 1/\delta t$). The policy operates at a lower frequency f_π , while simulation and hardware run at higher frequencies $f_{\text{sim}} = N_{\text{sim}} f_\pi$ and $f_{\text{real}} = N_{\text{real}} f_\pi$. Holding the jerk action constant over

$$\delta t_\pi = N_{\text{sim}} \delta t_{\text{sim}} = N_{\text{real}} \delta t_{\text{real}},$$

we generate an up-sampled joint-velocity trajectory with different discretizations for simulation and real deployment (see Fig. 4). The joint velocity and acceleration are clipped to remain within their prescribed limits, $\dot{q}_{\min} \leq \dot{q} \leq \dot{q}_{\max}$ and $\ddot{q}_{\min} \leq \ddot{q} \leq \ddot{q}_{\max}$.

4) *Reward Formulation*: The reward function encourages fast and accurate throwing and comprises live-time, dense shaping, sparse success, and regularization terms.

- *Live-Thrown* ($r_L = i_{\text{live},t} i_{\text{thrown},t}$): During early training, these two binary terms dominates, encouraging longer episodes ($i_{\text{live},t}$) and early object release ($i_{\text{thrown},t}$). Using only $i_{\text{live},t}$ led to unnecessary object holding.

- *Position Error* ($r_p = \|\mathbf{p}_{O,t} - \mathbf{p}_{\text{target}}\|_2$): This dense term is crucial for guiding initial exploration, which stalls when only sparse rewards are used. The error is the ℓ_2 distance between the object position $\mathbf{p}_{O,t}$ and the target $\mathbf{p}_{\text{target}}$.

- *Velocity to Target* ($r_{v_{\text{goal}}} = \left\langle \frac{\mathbf{v}_{O,t}}{\|\mathbf{v}_{O,t}\|_2}, \frac{\mathbf{p}_{\text{target}} - \mathbf{p}_{O,t}}{\|\mathbf{p}_{\text{target}} - \mathbf{p}_{O,t}\|_2} \right\rangle$): This term encourages movement toward the target via the dot product between the normalized object velocity and the normalized direction-to-target vector.

- *Success* ($r_{\text{success}} = i_{\text{success},t} \mathcal{K}(r_p; a)$): This is the primary objective for the bin-throwing task. The binary success indicator $i_{\text{success},t}$ is 1 when the object is within a 0.25 m threshold (half the real basket width) of the target. Accuracy is further encouraged by a smooth kernel $\mathcal{K}(r_p; a) = \frac{1}{e^{ar_p} + e^{-ar_p}}$, which increases exponentially as r_p decreases.

- *Regularization Penalties*. To obtain smooth and efficient trajectories, we penalize joint acceleration $r_{\ddot{q}} = \|\ddot{q}_t\|_2$ and jerk $r_{\dddot{q}} = \|\dddot{q}_t\|_2$. To damp motion post-release, we heavily penalize joint velocity after the throw via $r_{\dot{q}} = \|\dot{q}_t\|_2 \cdot i_{\text{thrown},t}$. These penalties are scaled relative to the mean of r_{success} avoiding excessive regularization early in training phases.

The total reward is a weighted sum of the above terms, with empirically-tuned weights ($w_L = 10$, $w_p = -1$, $w_{v_{\text{goal}}} = 10$, $w_{\text{success}} = 100$, $w_{\ddot{q}} = -100$, $w_{\ddot{q}} = -0.1$, $w_{\ddot{q}} = -0.01$).

5) *Episode Termination*: During training, episodes are terminated early upon failure to prevent low-quality data from entering the training buffer. Failure includes the object contacting the ground ($p_{O,t}^z < 0$). A time-limit termination is also applied at the fixed horizon.

6) *Implementation Details*: We use the NVIDIA IsaacLab framework [21] for efficient parallel data collection in NVIDIA Isaac Sim and train the policy with RL-Games [22] using Proximal Policy Optimization (PPO) [23]. Our physical platform is a Universal Robots UR5e (maximum reach < 1 m), operated up to an end-effector velocity of 5 m/s (with safety limits disabled). Control is issued via `ur-rtde` [24], a Real-Time Data Exchange wrapper for UR robots.

The policy is a feed-forward MLP with five hidden layers [1024, 512, 256, 128, 64] and GeLU activations. The trajectory horizon is 3.2 s, corresponding to 64 steps at a 20 Hz policy frequency. The up-sampled velocity trajectory is tracked at 120 Hz in simulation and 500 Hz on hardware (UR5e maximum). Additional architectural details are provided in our code release. Training across 4096 environments for 1000 epochs (about 2.62×10^8 steps) took a little over one hour on a single NVIDIA RTX 4090 GPU notebook.

B. Sim-to-Real Transfer

The simulation-trained policy can be reliably transferred to real hardware by minimizing the sim-to-real gap in both robot and object dynamics.

1) *Robot Dynamics Gap*: Prior to policy training, we ensure that commanded joint-velocity trajectories produce consistent behavior across simulation and hardware. To minimize this robot dynamics gap, we execute a set of minimum-jerk cyclic trajectories with varying amplitudes and frequencies on the physical system and record the resulting

joint positions and velocities. The same trajectories are then executed in simulation across N parallel environments (4096), each with different actuator stiffness and damping parameters. The optimal parameters are selected by minimizing a weighted sum of joint-position and joint-velocity errors between the simulated and recorded real trajectories. A second refinement stage samples additional parameter sets in a neighborhood around the optimized values to further reduce the error.

2) **Object Dynamics Gap:** Unlike the robot dynamics gap—which is minimized once—the object dynamics must be adjusted for each new object to maximize success. Although parameters such as size, mass, and static friction can often be estimated with reasonable accuracy, these estimates may still lead to failures due to the sim-to-real gap. To minimize the required tuning effort, we follow the steps below:

- 1) We conduct a sensitivity analysis (Fig. 7) to identify object properties most sensitive to modeling errors. This analysis reveals strong robustness to mass uncertainty but high sensitivity to dynamic-friction errors.
- 2) We train uncertainty-aware policies that explicitly account for friction-model estimation errors (Fig. 8).
- 3) For each object, we minimally tune only the dynamic-friction coefficient to maximize success at a chosen throwing target, and then reuse that value across all remaining targets.

Thanks to the robustness of the uncertainty-aware policy, we empirically find that a single object model can apply to multiple objects while maintaining high success rates.

V. EVALUATION

We first conducted extensive simulation experiments to evaluate the policy’s in-distribution *performance* (Fig. 5) and its out-of-distribution *generalization* (Table II). We performed *ablation* studies (Fig. 6) to assess the impact of key design choices. Prior to real-world deployment, we carried out a *sensitivity* analysis (Fig. 7) to evaluate robustness to modeling errors and then trained an *uncertainty-aware* policy (Fig. 8) to mitigate sensitivity to dynamic-friction mismatch. Finally, real-world evaluation across five objects and twelve throwing targets achieved an average success rate of 97% (Table III).

Note regarding baselines: Due to the absence of open-source implementations for model-based non-prehensile throwing, we could not perform fair comparisons against such methods. Using the “Upright” implementation [20]—originally developed for non-prehensile transportation—we attempted dynamic-grasping-based throwing. While partial success was achievable, solver failures were frequent and required careful tuning of the initial configuration, release pose/velocity, and solver hyperparameters (e.g., horizon length, cost weights, slack variables). Combining the strengths of RL and MPC represents an interesting direction for future work.

A. Simulation Results

1) **Performance Analysis:** We evaluated policy performance using 12,288 configurations (4096 per seed across three random seeds), sampled according to Table I. The average

success rate was approximately 99%. Fig. 5 shows how performance varies with different configuration parameters.

Performance degradation was observed primarily for extreme configurations that rendered the throw physically infeasible. For instance, when the robot starts with the first two links nearly vertical, the end-effector cannot generate sufficient acceleration to reach distant and elevated targets—particularly for objects with high height-to-width ratios. While object mass had limited influence on performance, higher static friction slightly reduced success rate. Empirically, we observed that sliding tends to dominate the motion, and larger friction coefficients increase the required end-effector accelerations to overcome frictional resistance. These findings suggest that surfaces with lower static and dynamic friction are advantageous for real-world deployment of our policy.

2) **Policy Generalization:** Beyond the training distribution, we evaluated the policy’s generalization to unseen objects and throwing configurations, as summarized in Table II. When evaluating cylindrical objects, we observed only a minor drop in performance, suggesting that both cuboids and cylinders exhibit similar contact behavior under our planar throwing policy. Our selection of YCB objects [25] included cuboid (“wood block”), cylindrical (“chips”), and generic shapes (“power drill”), all achieving high success rates. Performance degradation primarily occurred for configurations that are physically infeasible for the robot. Objects with large height-to-width ratios tend to roll or violate anti-roll constraints, particularly for distant or elevated targets. For unseen object models, the policy adapts reasonably well to lower masses and heights. Very low friction causes rapid sliding and insufficient object acceleration, hindering long-distance throws. Conversely, high mass or high friction induces larger friction forces, delaying sliding and limiting the achievable gravito-inertial acceleration. For tall objects, satisfying anti-rolling constraints while producing high acceleration may not be feasible. Finally, limits on maximum joint velocity, acceleration, and jerk impose corresponding limits on the set of reachable target positions for a given object model.

3) **Ablation Studies:** For each ablated design choice, we trained a policy for 500 epochs using three random seeds and evaluated it over 4096 in-distribution configurations. Fig. 6 compares success rates across the different design axes. In general, policy optimization becomes more challenging as the observation space size (history length), episode length (control rate), or action-space dimensionality (degrees of freedom) increases. Larger input spaces slow convergence and reduce success rates under a fixed compute budget (500 epochs). Using only the most recent joint state $\mathbf{x}_t$ provided sufficient observability while keeping the policy input compact. While higher control rates increase temporal resolution, they also lengthen the episode horizon, making credit assignment more difficult. Conversely, too low a control rate (e.g., 10 Hz) restricts the feasible trajectory space. Similarly, a low jerk limit cannot generate sufficient acceleration for throwing, whereas a large jerk limit requires more exploration to discover optimal actions. Lower jerk limits are also preferable in real-world deployment due to

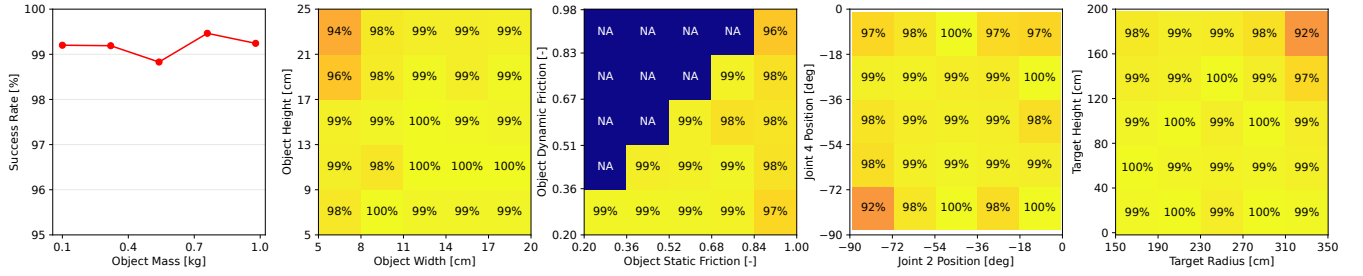


Fig. 5: Performance analysis of *NP-Throw* policy for in-distribution configurations. The success rate is shown as a line plot for object mass and as 2D histograms for pairs of configuration variables. Entries with dynamic friction higher than static friction are not applicable (N/A). The policy achieves consistently high success rates except under extreme combinations of object size, joint state, and throwing target.

TABLE II: Evaluation of policy generalization to unseen objects and throwing configurations. The policy generalizes well to cylindrical objects and YCB items [25], while performance degrades for infeasible throwing configurations. Success rates below 75% are highlighted in red. The “-Short” label denotes aligning the object’s short edge along the x -axis, increasing its height-to-width ratio.

Object	S.R. [%]	Object	S.R. [%]
Cuboids [Training]	98.97	Cylinders	98.54
YCB Objects			
WoodBlock	99.83	Chips	97.68
Spam	99.88	Pitcher	99.51
CrackerBox	99.90	CrackerBox-Short	96.44
MustardBottle	99.85	MustardBottle-Short	87.04
BleachCleanser	99.44	BleachCleanser-Short	79.49
PowerDrill	95.65	PowerDrill-Short	89.28
Object Model			
Mass [0.01–0.1] kg	98.97	Mass [1–3] kg	71.92
Friction [0.01–0.2]	87.06	Friction [1–3]	53.93
Height [1–5] cm	96.44	Height [25–40] cm	79.22
Target Position			
Elevation [2–2.5] m	81.54	Elevation [2.5–3] m	30.83
Distance [3.5–4] m	71.63	Distance [4–4.5] m	13.13

safer motions. Using all 6 DOF of the robot showed no benefit in our planar throwing task and introduced additional action dimensions that slightly reduced performance. The unexpectedly poor performance of a 2-DOF system is an interesting finding and warrants further investigation. Finally, both jerk-based and acceleration-based actions can induce the necessary contact-mode transitions, whereas velocity-based actions lead to more aggressive switching. While learning at high control rates with velocity actions may produce smoother trajectories, it incurs significantly higher computational cost.

4) *Policy Sensitivity to Modeling Uncertainty:* Estimating the true object model is nontrivial for real-world deployment due to the sim-to-real dynamics gap. To minimize the required tuning effort, we evaluate the policy’s sensitivity to modeling errors. For each component of β (e.g., mass), we fix the remaining parameters to their true values and randomly sample an estimate of that component from the distributions in Table I. Aggregated results over 4096 configurations and three random seeds are shown in Fig. 7.

When perturbing the modeled mass relative to the true mass, the policy maintains near-perfect success rates, indicating insensitivity to mass uncertainty. From Eqs. (1), (2), and (3), the anti-slip and anti-roll conditions are $\mu F_n \geq |F_t|$ and $w F_n \geq |\tau|$. Substituting the contact-wrench terms F_n , F_t , and τ with their gravito-inertial equivalents shows that the

mass m cancels in both inequalities. Accordingly, provided the end-effector tracks the commanded jerk/velocity trajectory accurately, the object experiences the same contact modes and release state. Nevertheless, the required joint torques do scale with mass and are bounded by hardware limits.

Object-size accuracy is more critical for thin (small true width) and tall (large true height) objects, for which anti-rolling constraints are harder to satisfy under uncertainty. The policy is moderately robust to errors in static friction but highly sensitive to dynamic friction: except for extremely high true static friction, underestimating friction tends to produce trajectories that overcome sticking; however, the release state under sliding depends strongly on the dynamic friction coefficient. Unlike prior work [5, 7, 8] that emphasizes dynamic grasping (more sensitive to static friction), our policy predominantly leverages sliding contact, making dynamic friction the key factor for accurate release.

Given this sensitivity, we train uncertainty-aware policies before real deployment. For each episode, we add a uniform noise ($\pm\nu$) to the true static and dynamic friction values. Fig. 8 compares success rates for policies trained under increasing friction uncertainty. The policy trained with perfect models (“None”) exhibits a sharper performance drop as uncertainty grows, whereas uncertainty-aware policies degrade more gracefully. Conversely, training with overly large noise (“Full”) prevents learning effective throws except for nearby targets. The friction-sensitivity plots further show improved robustness for uncertainty-aware training, which is preferable in practice.

B. Real-World Evaluation

Evaluation Setup. We selected five objects with distinct physical properties as illustrated in Fig. 9. Throwing targets consisted of four distances (200, 250, 300, 350 cm) and three elevations (15, 105, 185 cm), resulting in 60 combinations ($5 \times 4 \times 3$). For each configuration, the policy was first evaluated in simulation across 64 randomized initial joint configurations; the first 10 successful trajectories were then executed on hardware. Following simulation findings (Fig. 5), we placed a sheet of paper on the tray to reduce the static friction introduced by the original rubber surface. The static and dynamic friction coefficients were tuned empirically using the wood block at the (250 cm, 105 cm) target, resulting in $\mu_s = 0.5$ and $\mu_k = 0.2$, which were applied uniformly across all objects and targets. Leveraging the policy’s robustness to mass-model errors (Fig. 7), we approximated all object

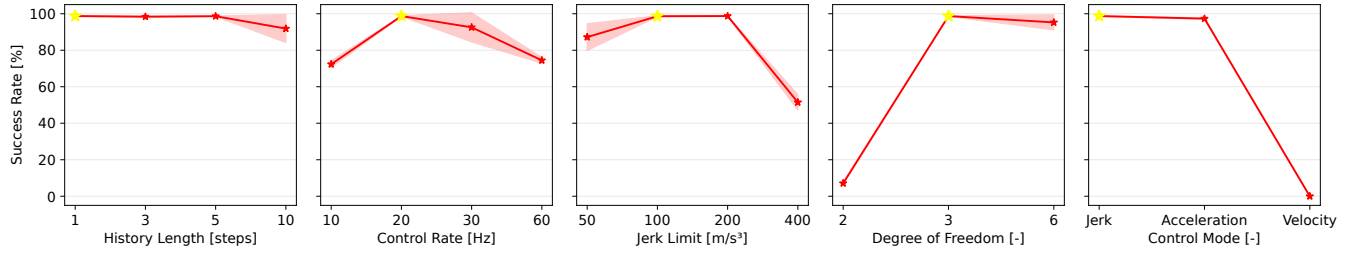


Fig. 6: Ablation study across different design axes. Success rates (mean and standard deviation) are computed over 4096 configurations and 3 random seeds. Our proposed policy configuration achieves the highest success rate, highlighted by a yellow star.

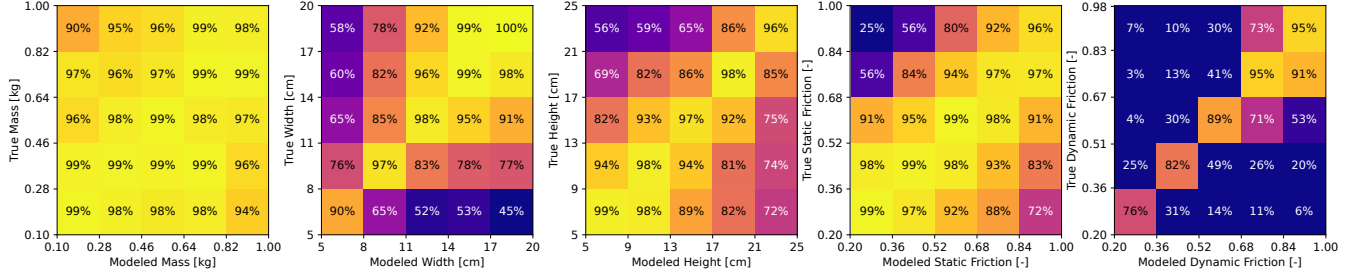


Fig. 7: Evaluation of policy robustness to modeling uncertainties, measured by success rate. As the estimated object model (x-axis) deviates from the true model (y-axis), performance is minimally affected by mass uncertainty but is most sensitive to dynamic-friction errors.

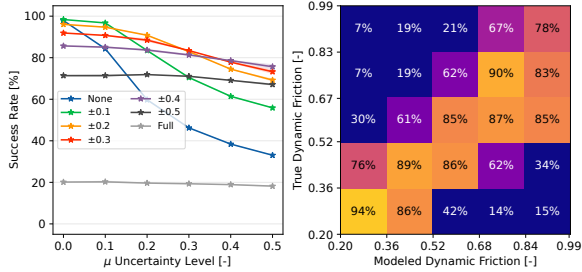


Fig. 8: Left: Performance of uncertainty-aware policies starting from zero uncertainty training (“None”) to fully random (“Full”) for different levels of friction uncertainty. Right: Sensitivity to dynamic friction for the policy trained with ± 0.3 friction noise.

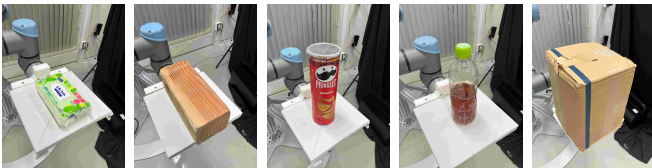


Fig. 9: Real-world evaluation objects: *deformable* tissue box (#1), *heavy* 790 g wood block (#2), *tall cylindrical* chips (#3), *partially filled* bottle (#4), and *large* 20 × 20 × 28 cm box (#5).

masses as 0.5 kg. Object size was estimated using an axis-aligned bounding box. For robust deployment, we used the policy trained with ± 0.3 friction uncertainty (Fig. 8).

Results. Table III reports an average success rate of 97% across the 60 evaluated combinations (excluding the invalid configuration). At the (350 cm, 185 cm) target, all objects required release velocities that caused the policy-generated end-effector trajectory to dip too low—an artifact of the robot stand not being modeled during training—leading to collisions. In addition, the maximum end-effector velocity (M.EE.V.) exceeded the hardware limit (5 m/s), making real deployment unsafe. For other extreme targets (e.g., (300 cm, 185 cm)),

TABLE III: Real-world evaluation for the objects in Fig. 9. The policy achieves an average **97%** success rate across all objects and targets. Extreme targets (e.g., 350 cm distance at 185 cm elevation) failed (marked **F**) due to infeasible end-effector velocities and collisions with the robot base. We also report aggregated statistics ($\mu \pm \sigma$) for the maximum end-effector velocity (M.EE.V.) and throwing time (Throw T.) for each target.

r_t [cm]	z_t [cm]	Real Success Rate [10]					M.EE.V. [m/s]	Throw T. [ms]
		#1	#2	#3	#4	#5		
200	15	10	10	10	10	10	3.45±0.02	614±39
	105	10	10	10	10	10	3.87±0.07	701±49
	185	10	10	10	10	10	4.53±0.05	1141±50
250	15	10	9	9	10	10	4.05±0.09	745±68
	105	10	10	10	10	10	4.07±0.06	1005±57
	185	10	10	10	9	10	4.70±0.04	1263±63
300	15	10	8	9	10	10	4.10±0.07	1112±64
	105	10	10	10	9	8	4.39±0.02	1178±44
	185	10	10	F	F	F	4.97±0.04	1390±59
350	15	9	10	10	9	10	4.42±0.04	1265±51
	105	10	8	F	F	F	4.62±0.02	1226±48
	185	F	F	F	F	F	5.18±0.04	1455±50

feasible trajectories could not be generated for high height-to-width ratio objects (e.g., the chips can). Across the remaining targets, failures were primarily due to small placement inaccuracies (object not perfectly centered on the tray) or modeling errors that affected release timing—effects that are amplified at long distances. The average throwing duration was approximately 1 s, and when combined with efficient picking (e.g., conveyor-based feeding), the system demonstrates high throughput potential.

Note on policy robustness. The simulation robustness trends in Fig. 7 were validated on hardware: trajectories optimized for a single object could be reused across a wide range of other objects while maintaining high success rates. As illustrated in Fig. 1, two distinct objects could even be thrown simultaneously using the same planned trajectory.

VI. CONCLUSION

We presented a reinforcement-learning perspective on non-prehensile throwing that addresses key limitations of model-based approaches. In simulation, we extensively evaluated performance, generalization, and sensitivity, identifying dynamic friction as the pivotal parameter for our predominantly sliding-based release. With uncertainty-aware training, the simulation-trained policy transferred to hardware, achieving an average success rate of 97%.

Limitations & Future Work. Beyond planar throwing, extending to full 3D throwing [3] and increasing control DoF [2] could yield more flexible strategies, particularly in cluttered environments. Rather than relying on manual and imprecise object modeling—especially for friction—future work will investigate autonomous dynamics identification with an emphasis on minimizing the sim-to-real gap. Another two complementary directions are (i) improving robustness to generic objects with irregular shape, nonuniform density, or non-planar contact, and (ii) increasing throwing precision for known object models. Finally, because heavy, long-distance throws can induce high-impact collisions, impact-aware throwing and complementary catching policies [26] are important for safer deployment.

REFERENCES

- [1] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser, “Tossingbot: Learning to throw arbitrary objects with residual physics,” 2019.
- [2] Y. Liu and A. Billard, “Tube acceleration: Robust dexterous throwing against release uncertainty,” *IEEE Transactions on Robotics*, vol. 40, pp. 2831–2849, 2024.
- [3] L. Werner, F. Nan, P. Eyschen, F. A. Spinelli, H. Yang, and M. Hutter, “Dynamic throwing with robotic material handling machines,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.19001>
- [4] M. Bombile and A. Billard, “Bimanual dynamic grabbing and tossing of objects onto a moving target,” *Robotics and Autonomous Systems*, vol. 167, p. 104481, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0921889023001203>
- [5] K. Lynch and M. Mason, “Dynamic underactuated nonprehensile manipulation,” in *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS '96*, vol. 2, 1996, pp. 889–896 vol.2.
- [6] J. Z. Woodruff and K. M. Lynch, “Planning and control for dynamic, nonprehensile, and hybrid manipulation tasks,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 4066–4073.
- [7] A. Pekarovskiy and M. Buss, “Optimal control goal manifolds for planar nonprehensile throwing,” in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2013, pp. 4518–4524.
- [8] H. Miyashita, T. Yamawaki, and M. Yashima, “Parts assembly by throwing manipulation with a one-joint arm,” in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2010.
- [9] W. Mori, J. Ueda, and T. Ogasawara, “1-dof dynamic pitching robot that independently controls velocity, angular velocity, and direction of a ball: Contact models and motion planning,” in *2009 IEEE International Conference on Robotics and Automation*, 2009, pp. 1655–1661.
- [10] H. Miyashita, T. Yamawaki, and M. Yashima, “Control for throwing manipulation by one joint robot,” in *2009 IEEE International Conference on Robotics and Automation*, 2009, pp. 1273–1278.
- [11] Y. Liu, A. Nayak, and A. Billard, “A solution to adaptive mobile manipulator throwing,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 1625–1632.
- [12] S. Kim, A. Coninx, and S. Doncieux, “From exploration to control: Learning object manipulation skills through novelty search and local adaptation,” *Robotics and Autonomous Systems*, vol. 136, p. 103710, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0921889020305509>
- [13] Y. Liu, B. D. Costa, and A. Billard, “Learning to throw-flip,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.10357>
- [14] W. Mori, J. Ueda, and T. Ogasawara, “Differentiable motion manifold primitives for reactive motion generation under kinodynamic constraints,” in *2009 IEEE International Conference on Robotics and Automation*, 2025, pp. 1655–1661. [Online]. Available: <https://arxiv.org/abs/2410.12193>
- [15] C. Chu and H. Xu, “Da-mmp: Learning coordinated and accurate throwing with dynamics-aware motion manifold primitives,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.23721>
- [16] M. Monastirsky, O. Azulay, and A. Sintov, “Learning to throw with a handful of samples using decision transformers,” *IEEE Robotics and Automation Letters*, vol. 8, no. 2, pp. 576–583, 2023.
- [17] S. Aslam, K. Kumar, P. Zhou, H. Yu, M. Yu Wang, and Y. She, “Dartbot: Overhand throwing of deformable objects with tactile sensing and reinforcement learning,” *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 13 644–13 661, 2025.
- [18] H. Kasaei and M. Kasaei, “Throwing objects into a moving basket while avoiding obstacles,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 3051–3057.
- [19] A. Petrenko, A. Allshire, G. State, A. Handa, and V. Makoviy-chuk, “Dexpb: Scaling up dexterous manipulation for hand-arm systems with population based training,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [20] A. Heins and A. P. Schoellig, “Keep it upright: Model predictive control for nonprehensile object transportation with obstacle avoidance on a mobile manipulator,” *IEEE Robotics and Automation Letters*, vol. 8, no. 12, pp. 7986–7993, 2023.
- [21] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, and A. Garg, “Orbit: A unified simulation framework for interactive robot learning environments,” *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.
- [22] D. Makoviichuk and V. Makoviychuk, “Rl games,” 2021. [Online]. Available: https://github.com/Denys88/rl_games/
- [23] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017.
- [24] A. P. Lindvig, I. Iturrate, U. Kindler, and C. Sloth, “ur-rtde: An interface for controlling universal robots (ur) using the real-time data exchange(rtde),” in *2025 IEEE/SICE International Symposium on System Integration (SII)*, 2025, pp. 1118–1123.
- [25] B. Calli, A. Walsman, A. Singh, S. Srinivasa, P. Abbeel, and A. M. Dollar, “Benchmarking in manipulation research: Using the yale-cmu-berkeley object and model set,” *IEEE Robotics Automation Magazine*, no. 3, 2015.
- [26] A. Pekarovskiy, F. Stockmann, M. Okada, and M. Buss, “Hierarchical robustness approach for nonprehensile catching of rigid objects,” in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014, pp. 3649–3654.